

AI-Powered Cartoon Character Storyline Generator — Agentic Web App

A LangGraph multi-agent system with a Streamlit UI, PostgreSQL persistence, and Langfuse/LangSmith observability.

Estimated effort: 3–5 hours | Format: Take-home, build with any AI coding agent (Cursor, Claude Code, Copilot, etc.) | Level: Backend / AI Engineer

1 Overview

You will build a small but complete **agentic web application**. A user describes a new cartoon character through a guided brief (free text + structured dropdowns), and a **LangGraph** pipeline of two cooperating agents — a **Storyline Generator Agent** and a **Review/Critic Agent** — produces a character storyline that is verifiably aligned with the brief. The review agent should not be a rubber stamp: it must actually critique the draft against the brief and send it back for revision when it isn't aligned, in a visible generate → review → revise loop.

This assignment is intentionally scoped to be buildable in 3–5 focused hours using an AI coding agent. We are evaluating your judgement on **agent orchestration, state design, prompt/quality control, and engineering hygiene** — not visual polish.

2 Problem Statement

Build "**CharacterForge**": a Streamlit app where a studio writer can describe a new cartoon character and receive a short, structured storyline (origin, personality beats, world, a 3–5 scene arc, and a tagline) that a review agent has verified is faithful to the brief before it is shown as final.

2.1 User Input Flow (Streamlit)

The UI must collect a complete brief across these fields before generation is allowed:

- **Character brief** (free text, required) — how the character should look, e.g. "a small round robot with mismatched mechanical wings and a cracked glass chest panel."
- **Tone / mood** (dropdown, required) — e.g. Emotional, Funny, Energetic, Mysterious, Heroic, Calm.
- **Target audience** (dropdown, required) — e.g. Toddlers (2–5), Kids (6–11), Tweens (12–15), All ages.
- **Show format** (dropdown, required) — e.g. Short-form (2–5 min), Episodic series (11–22 min), Feature film.
- **Setting / world** (dropdown or short text, required) — e.g. Fantasy kingdom, Space colony, Modern city, Underwater world, Post-apocalyptic.
- **Core trait / power** (short text, optional) — a defining ability, quirk, or flaw.
- **Number of scenes** (slider or dropdown, optional, default 4) — length of the generated arc.

The **Generate Storyline** button should stay disabled (or clearly validate) until all required fields are filled — this is the "once all required details are asked" gate from the brief.

2.2 Agentic Pipeline (LangGraph)

Model the workflow as a LangGraph state graph with at least these nodes:

- **Input Validator node** — normalizes the collected brief into a structured state object.
- **Storyline Generator Agent** — drafts the storyline (character summary, personality, world, scene-by-scene arc, tagline) strictly from the brief + tone + audience + format.
- **Review / Critic Agent** — independently scores the draft against the original brief on concrete criteria (tone match, visual/brief fidelity, audience appropriateness, scene count, internal consistency) and returns either APPROVED or NEEDS_REVISION with itemized, actionable feedback.
- **Conditional edge** — routes back to the Generator with the critic's feedback appended to state if NEEDS_REVISION, or forward to the end if APPROVED or a max-iteration cap (e.g. 3) is hit.
- **Finalizer node** — persists the final storyline + full revision history to Postgres.

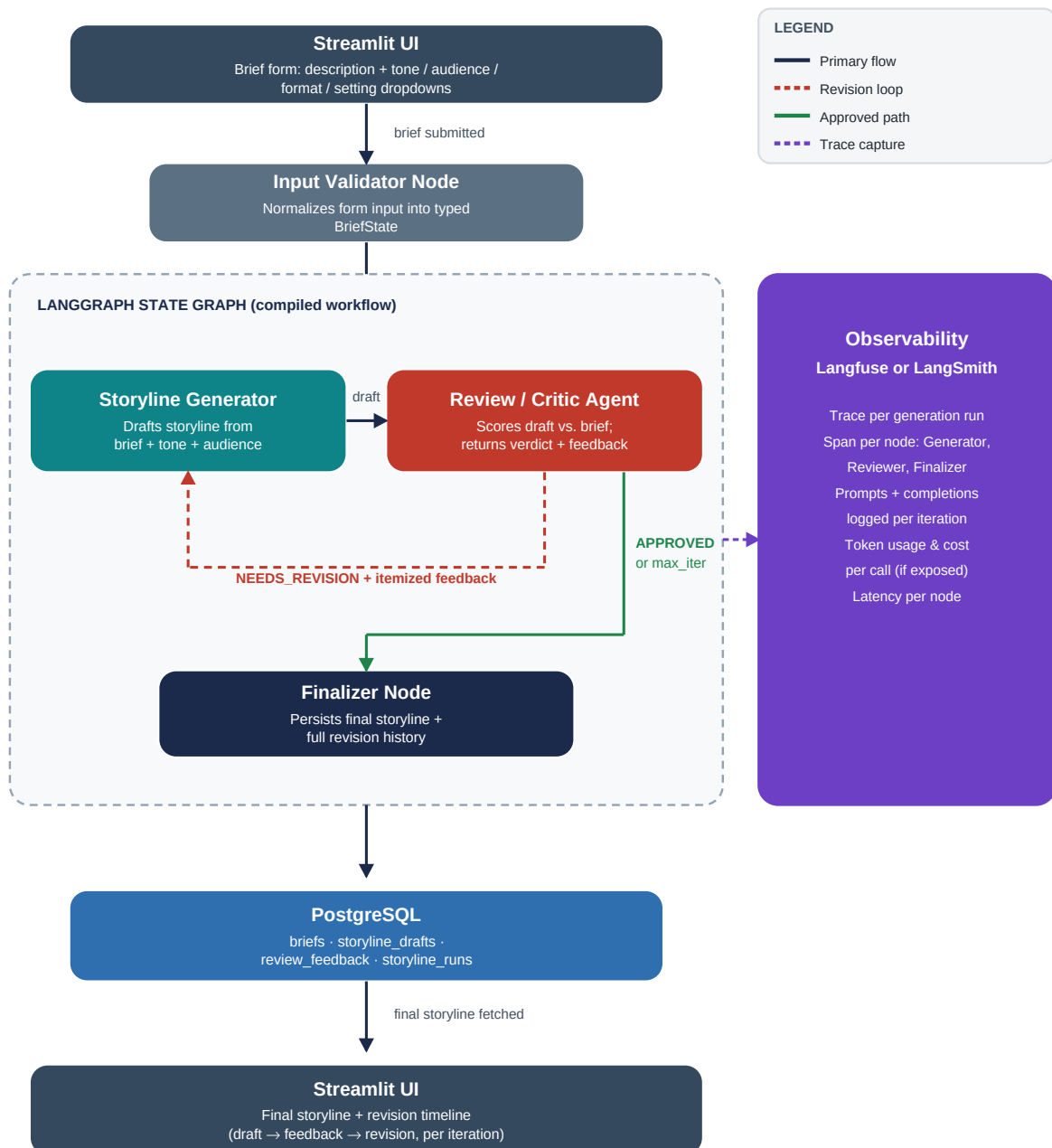
Each iteration (draft, critique, revision) must be distinguishable in your logs / DB / UI — this loop *is* the core of the assignment, not an add-on.

3 System Design You Must Produce

As part of your submission, document your own design decisions — do not just submit code.

3.1 Architecture / Flow Diagram

Provide a diagram (draw.io, Excalidraw, Mermaid, or hand-drawn is fine) showing: Streamlit UI → Input Validator → LangGraph state graph with all nodes and conditional edges → Postgres persistence → observability tracing hooks. A reference architecture is shown below — you are free to diverge from it and should explain why in your design note.



The observability panel (right) is intentionally decoupled from the state-graph container: whichever tool you choose should capture a trace per generation run with one span per node (Generator, Reviewer, Finalizer), including prompts, completions,

token counts, latency, and cost where the provider exposes it.

3.2 Coding Standards Note

Include a short CODING_STANDARDS.md covering the choices you made and why, e.g.:

- Project/module layout (agents, graph, db, ui, prompts separated).
- How LangGraph state (TypedDict / Pydantic model) is typed and versioned across nodes.
- Prompt management approach (templates, system vs. user messages, where they live).
- Error handling: LLM timeouts, malformed JSON from the critic, max-iteration cutoffs.
- Testing approach, even if minimal (e.g. unit test for the routing/conditional-edge logic).
- Secrets/config handling (.env, no hard-coded keys).

4 Required Tech Stack

Layer	Requirement
Orchestration	LangGraph (Python) for the generator <-> reviewer agent loop
UI	Streamlit — brief form, dropdowns, generate button, results + revision history view
LLM Provider	Any (OpenAI, Anthropic, etc.) — pick one and note the model used
Database	PostgreSQL preferred (SQLite acceptable as a documented fallback)
Observability	Langfuse or LangSmith (self-hosted or cloud, free tier) — required. Trace every agent call, iteration, prompt/completion, token count, and cost. This is weighted heavily in evaluation (see Section 7).
Language	Python 3.10+

Pick one: Langfuse or LangSmith. Both are built for exactly this — tracing LLM/agent calls with prompts, completions, token usage, latency, and cost. Langfuse is open-source (self-host via Docker Compose or use their free cloud tier) and integrates with LangGraph via a callback handler (`langfuse.langchain.CallbackHandler`) or native OpenTelemetry-based tracing. LangSmith is LangChain's own hosted tracing platform and integrates natively with LangGraph with minimal setup (a few environment variables). Read whichever you pick's docs and wire it in so every node execution, prompt, and token/cost figure shows up as a trace per generation request — this observability trace is one of the most heavily weighted parts of this assignment (40%, see Section 7).

5 Suggested Data Model (Postgres)

```
briefs(id, description, tone, audience, format, setting, core_trait,
       scene_count, created_at)

storyline_runs(id, brief_id, status, iteration_count,
               final_storyline_id, created_at)

storyline_drafts(id, run_id, iteration, content_json,
                  generated_by, created_at)

review_feedback(id, run_id, iteration, verdict,
                 criteria_scores_json, feedback_text, created_at)
```

Feel free to normalize/denormalize differently — the key requirement is that every draft and every review verdict + feedback text for a run is retrievable and traceable after the fact.

6 Deliverables

Submit all of the following:

- **1. Working code** on a public (or invite-shared) GitHub repo, with a clear README (setup, env vars, how to run Streamlit + Postgres + Langfuse/LangSmith locally).
- **2. System design flow diagram** (Section 3.1) + a short **coding standards** note (Section 3.2), included in the repo (e.g. /docs).
- **3. At least 2 example runs** showing a storyline that was *not* aligned with the brief on the first pass, the review agent's specific feedback, and the corrected final storyline — via logs, DB export, or the UI's revision-history view.
- **4. Observability evidence (heavily weighted)**: a screenshot or short recording of your Langfuse or LangSmith dashboard for this project, showing traces per run, per-node spans (Generator / Reviewer / Finalizer), and token/cost figures (if your provider exposes cost). This is one of the most important deliverables — see the 40% weight in Section 7.
- **5. Streamlit UI**, runnable locally at minimum; a deployed link (Streamlit Community Cloud, Render, etc.) is optional but appreciated.
- **6. A short screen recording (5–10 min)** walking through: filling the brief, the generate→review→revise loop happening live, the final storyline, and the Langfuse/LangSmith dashboard — *required if you skip deployment*, optional but recommended either way.

7 Evaluation Criteria

Observability (Langfuse/LangSmith) is the single most heavily weighted area of this assignment at 40%. A correct generator/reviewer loop that cannot be inspected — trace by trace, node by node, with prompts, tokens, latency, and cost — is treated as incomplete. Wire up tracing early, not as an afterthought at the end.

Area	What we're looking for	Weight
Agentic design	Clean LangGraph state graph; real conditional routing on the critic's verdict, not a fixed pipeline; sensible max-iteration guard	25%
Review agent quality	Critic checks concrete, brief-derived criteria and gives actionable feedback the generator visibly acts on	15%
Observability — Langfuse or LangSmith	Every node (Generator, Reviewer, Finalizer) produces a real trace/span per run; prompts, completions, tokens, latency, and cost (if exposed) are all visible on the dashboard; Postgres persistence backing it is sensible and queryable	40%
Code quality	Structure, typing, error handling, secrets hygiene, README clarity	10%
UI/UX	Form validation, clear display of final storyline and revision history	5%
Documentation	Flow diagram and coding-standards note clearly explain the 'why'	5%

8 Notes & Constraints

- You're encouraged to use an AI coding agent (Cursor, Claude Code, Copilot, etc.) — that's part of the exercise. We care about the decisions you made and can defend, not hand-typed code.

- Keep scope tight: two agents (generator + reviewer), one revision loop, one entity (the cartoon character). Don't build multi-character or multi-episode support.
- If you can't get Postgres running locally in time, SQLite with a note in the README is fine — just say so.
- Either Langfuse or LangSmith is acceptable — pick whichever you're faster with. If Langfuse cloud sign-up is inconvenient, self-hosting via their Docker Compose is fine too; LangSmith's free tier works the same way with just environment variables.
- If you get stuck or run short on time, ship what you have and document what's missing/next in the README rather than going silent.

Questions about scope are part of the exercise — use your best judgement and document any assumptions you made in the README.

Good luck — we're looking forward to seeing how you think, not just what you ship.